

470万阅读！前亚马逊/迪士尼工程师的 Claude Code 实战心得

mp.weixin.qq.com/s/1fSp0JUqdrn-3RCAhzZxtA

阿杰



Original

前几天推特上有一篇《The complete claude code tutorial》的帖子火了

作者 Eyad 是一个7年的软件工程师，之前在亚马逊、迪士尼和 Capital One 工作过，现在是一家企业 Agent 初创公司的 CTO。

他发的这篇文章在推特上获得了超过 **470 万阅读**，引发了大量讨论。

我看了一下，确实有不少值得分享的实战经验，尤其是关于如何**系统化地使用 Claude Code** 而不是把它当成简单问答工具。

今天把核心要点整理出来，希望对你有用。

先思考，再打字

大多数人使用 Claude Code 的第一反应是直接开始打字或说话。但作者说，这可能是你一开始就会犯的最大错误。

按两次 Shift + Tab 进入 Plan Mode（规划模式），先和 Claude 讨论清楚架构再开始实现。

这听起来很基础，但作者测试了十次，Plan Mode 的输出质量都显著优于直接开始聊天。

如果你没有多年软件工程经验来独立思考这些问题，作者建议用两个办法：

1. 开始学习系统设计，哪怕每次只学一点点
2. 先和 ChatGPT/Gemini/Claude 深入讨论架构方案，和 LLM 相互提问而不是单向沟通

本质上就是：**先思而后行**。在 Plan Mode 中信息越充分，输出就越好，因为输入会更好。

CLAUDE.md

当你启动 Claude Code 时，Claude 做的第一件事就是读取你的 CLAUDE.md 文件。这个文件中的每条指令都会影响 Claude 如何处理你的项目。

但大多数人要么完全忽略它，要么塞满垃圾内容，让 Claude 变得更糟。

真正重要的原则：

保持简短。 Claude 一次只能可靠地遵循大约 150 到 200 条指令，而 Claude Code 的系统提示已经使用了其中约 50 条。

如果你的 CLAUDE.md 像小说一样长，Claude 会开始随机忽略内容。

针对你的项目具体化。 不要解释 components 文件夹是什么，Claude 知道组件是什么。告诉它那些真正重要的 Bash 命令和你工作流程中的特殊内容。

告诉它为什么，而不只是做什么。

"使用 TypeScript strict 模式"——还可以。"使用 TypeScript strict 模式，因为我们曾因隐式 any 类型在生产环境出现过 Bug"——更好。

这个"为什么"为 Claude 提供了上下文，让它能在你没有预料到的情况下做出判断。

持续更新它。 工作时按 # 键，Claude 会自动将指令添加到你的 CLAUDE.md。每次你发现自己在同一件事上纠正 Claude 两次，这就是一个信号：它应该被写进文件。

上下文窗口的局限性

这是一个很多人没意识到的关键点：模型在你达到 100% 上下文之前就已经开始退化了。

在大约 20-40% 的上下文使用率时，输出质量就开始下降，即使不明显。如果你遇到过 Claude Code 压缩后仍然给你糟糕的输出，这就是原因。

几个有效做法：

限定对话范围。 一次对话对应一个功能或任务。不要在同一个对话中既构建认证系统又重构数据库层。

使用外部记忆。 让 Claude 将计划和进度写入实际文件（作者用 SCRATCHPAD.md 或 plan.md）。这些会跨会话保留。

复制粘贴重置。 当上下文变得臃肿时，从终端复制所有重要内容，运行 `/compact` 获取摘要，然后 `/clear` 完全清除上下文，只粘贴回重要的内容。

知道何时清除。 如果对话已经偏离轨道，直接 `/clear` 并重新开始。

有效的心智模型：**Claude 是无状态的。** 每次对话都从零开始，除了你明确给它的内容。

Prompt 决定一切

人们花几周时间学习框架和工具，却不花时间学习如何与真正生成代码的东西沟通。

几个真正有帮助的做法：

对你想要的东西具体化。"构建一个认证系统"给了 Claude 创作自由，它会用得很糟糕。"使用这个现有的 User 模型构建邮箱/密码认证，将 Session 存储在 Redis 中，添加中间件保护 `/api/protected` 下的路由"给了 Claude 明确的目标。

告诉它不要做什么。Claude 4.5 特别喜欢过度设计——额外的文件、不必要的抽象。

如果你想要简单的东西，就说"保持简单。不要添加我没要求的抽象。如果可能的话，一个文件搞定。"

给它关于为什么的上下文。"我们需要保证这个的效率，因为它在每个请求上运行"——会改变 Claude 处理问题的方式。"这是一个我们会丢弃的原型"——会改变哪些权衡是合理的。

记住：**输出就是一切，但它只来自输入。**如果你的输出很烂，你的输入就很烂。

当 Claude 卡住时

有时候 Claude 就是会陷入循环，反复尝试同样的操作，失败了又重来。

遇到这种情况时，更有效的做法是**彻底更换思路**：

清空对话记录。输入 `/clear` 指令，开启全新的对话。

简化任务。如果 Claude 在复杂任务上挣扎，将其分解为更小的部分。

展示而不是告诉。自己写一个最小的示例，"这就是输出应该的样子。现在将这个模式应用到其余部分。"

尝试新思路。调整问题的表述方式，比如把"处理好这些状态转换"换成"把这个功能实现为一个状态机"。

核心技巧是**尽早识别出自己正在陷入无效循环**。如果你已经把同一个问题解释了三遍，Claude 还是没能理解，那再多的解释也是徒劳。

构建系统，而不是一次性任务

能从 Claude 身上榨取最大价值的人，从来不会只把它用来处理一次性任务。他们会搭建一套系统，让 Claude 成为其中的一个组件。

Claude Code 的代码功能自带一个 `-p` 参数，可用于启用无界面模式。在这种模式下，它会直接执行你输入的指令并输出结果，无需进入交互式操作界面。

这意味着你可以为它编写脚本，将它的输出内容传递给其他工具，与 Shell 命令串联使用，或是把它集成到自动化工作流中。

企业正利用这项功能实现诸多自动化场景：自动审核 PR、自动回复客户支持工单、自动完成日志记录与文档更新。所有操作都会留存日志，可供审计追溯，并且系统会根据实际使用中的成效不断迭代优化。

这里存在一个良性循环：Claude 出现失误后，你查看相关日志，然后优化 CLAUDE.md 文档或配套工具，下一次 Claude 的表现就会更出色。

总结

这篇文章的核心观点可以概括为：

先思而后行。Plan 模式规划产生的结果比直接开始说话好得多。

CLAUDE.md 文件。保持简短、具体，告诉它为什么，并持续更新。

上下文窗口的局限性。使用外部记忆（将计划和进度写入实际文件）、限定对话范围，不要害怕清除。

输出质量取决于输入质量。如果在使用好模型时得到了糟糕的结果，你的提示词输入需要改进，改进方向可以有：对你想要的东西具体化、告诉它不要做什么、给它关于为什么的上下文。

构建系统，而不是一次性任务。Claude 出现失误后优化更新 CLAUDE.md 文件，实现长期的迭代优化。

说下我自己的感受，我近期写代码比之前多用了 Plan 规划模式，先讨论完再一次性动手，确实比直接开始好多了。

如果你要做个完整的需求或小功能，我推荐也是先使用 Plan 模式，相当于需求澄清与设计技术实现方案。

AI 实战与副业变现。🌟 如果觉得文章有用，欢迎「关注 + 星标」，交个朋友，一起进化。